

Ava Stephens
May 8th, 2026
BINF 6970

TCGA Glioblastoma Database Project Writeup

This project built a relational database using TCGA Glioblastoma Multiforme data from cBioPortal. The main goal was to take several separate clinical and molecular data files and organize them into a structured MySQL database that could be queried more easily. The final database connects patient information, sample level clinical information, genes, mutations, mRNA expression values, copy number alteration values, protein expression values, protein mutation annotations, and sequencing panel information. Instead of leaving the data spread across multiple large spreadsheets, this project reorganizes the information into related tables with primary keys, foreign keys, and documented relationships.

The database was designed for someone who wants to explore TCGA GBM data in a structured way. For example, a user could query a sample's clinical subtype, connect that sample to mRNA expression values, identify copy number alteration values for specific genes, or connect mutations to genes and protein changes. The project also includes a Neo4j component, where differentially expressed genes gotten through SQL were loaded into a graph database. In that graph, genes are connected to the GBM subtype where they had higher mean expression. The final MySQL database has 13 tables and 3,340,503 total rows. The final tables are patient, cancer_type, sample, gene, mutation, mutation_sample, mutation_gene, copy_number, mrna_expression, protein, protein_quant, protein_mutation, and sequencing_panel. The final repository also includes Python scripts, SQL scripts, a SQL dump/export, a Neo4j/Cypher script, diagrams, and documentation files so that another person could understand and recreate the project.

Data Sources

The data for this project came from the TCGA Glioblastoma Multiforme study available through cBioPortal. The study link is:

https://www.cbioportal.org/study/summary?id=gbm_tcg_a_pub2013. The dataset includes 577 patients/samples overall, with whole exome and/or whole genome sequencing available for 291 of the 577 glioblastoma tumor/normal pairs. The data types used in the project included clinical patient data, clinical sample data, mutations, mRNA expression, copy number alterations, RPPA protein expression, and gene panel information.

Seven main source files were used. The clinical patient file provided patient level information, mainly patient identifiers and sex. The clinical sample file provided sample level information such as sample ID, patient ID, cancer type, sample type, expression subtype, MGMT status, methylation status, G-CIMP methylation, IDH1 mutation, tumor mutational burden, OncoTree code, and survival related fields. The mutation file provided mutation level

information, including gene symbols, Entrez gene IDs, chromosome, mutation positions, variant classifications, variant types, alleles, HGVS annotations, and protein changes. The mRNA expression file provided gene expression values in a matrix format, where genes were rows and samples were columns. The copy number alteration file also came in a matrix format, with genes as rows and samples as columns. The RPPA file had protein expression measurements, and the gene panel matrix file had information about whether samples were profiled for mutation and GISTIC/CNA data. Some files were too large to store directly in GitHub, so the raw files, cleaned files, full SQL population file, and full database dump were stored externally using Google Docs. The GitHub repository still includes links to those files, along with the scripts and documentation needed to understand and recreate the database.

Data Cleaning Process

The data cleaning process was mainly focused on making the source files consistent and usable for easy relational database loading. ChatGPT was used to help clean the original data files. Across the datasets, column names were standardized, cBioPortal metadata rows were removed when present, text fields were trimmed, and missing or unavailable values were converted into blanks so they could load as SQL NULL. Key identifiers were kept because they were needed to connect records across tables. These included TCGA patient IDs, TCGA sample IDs, HUGO gene symbols, Entrez gene IDs, mutation IDs, and protein IDs.

The clinical patient file originally contained 577 patient records and 8 columns. The first four cBioPortal metadata rows were removed because they were descriptions, data type labels, or priority/order rows rather than actual patient data. The cleaned patient level file kept patient_id and sex for the patient table, and sex values were standardized to Male and Female. All 577 patients were kept because each patient had a unique, non null patient ID. Survival status values were simplified by removing cBioPortal prefixes, so values like 1:DECEASED became DECEASED and 0:LIVING became LIVING. The THERAPY column was not kept because it was unnecessary information.

The clinical sample file also contained 577 records. Its first four cBioPortal metadata rows were removed, and the cleaned sample sheet kept fields needed for the sample table, including sample_id, patient_id, cancer_type_id, sample_type, expression_subtype, mgmt_status, methylation_status, g_cimp_methylation, idh1_mutation, oncotree_code, somatic_status, and tmb_non_syn. The original TMB_NONSYNONYMOUS column was renamed to tmb_non_syn and rounded to four decimal places. All sample rows were kept because each had a unique sample ID. Cancer type information was separated into its own cleaned cancer_type sheet instead of being repeated in every sample row. This created one normalized cancer type record for the TCGA GBM dataset and allowed cancer_type_id to be used as a foreign key in the sample table.

The mutation file required the most restructuring because it was a very wide annotation file with 16,325 mutation records and 281 columns. It included gene identifiers, sample

identifiers, mutation positions, variant classifications, alleles, HGVS annotations, protein IDs, and protein change annotations all in one file. To match the ER diagram, the mutation file was split into cleaned sheets for gene, mutation, mutation_sample, mutation_gene, protein, and protein_mutation.

The cleaned gene sheet from the mutation file created 8,237 unique gene records. Gene symbols were cleaned from Hugo_Symbol, and Entrez IDs were cleaned from Entrez_Gene_Id. Entrez IDs listed as 0 were converted to blank because 0 is not a meaningful Entrez identifier. Chromosome values were cleaned into consistent text values, and gene symbols that Excel had accidentally converted into dates were corrected, such as March 4 to MARCH4. Gene start position, gene end position, and cytoband were left blank when true gene level values were not available, because mutation coordinates are not the same as full gene coordinates.

The cleaned mutation sheet created 16,090 unique mutation records. Mutations were deduplicated based on biological mutation identity, including chromosome, start position, end position, variant classification, variant type, reference allele, tumor alternate allele, HGVSc, and HGVSp. Sample IDs were removed from the mutation table because the same mutation can appear in more than one sample. Instead, a mutation_sample bridge table was created with 16,325 mutation sample relationships. A separate mutation_gene bridge table was also created with 16,090 mutation gene relationships, which kept mutation and gene information connected without repeating gene details inside the mutation table.

Protein related mutation information was separated into protein focused tables. The cleaned protein sheet used the best available protein identifier from fields such as Refseq_prot_Id, ONCOTATOR_REFSEQ_PROT_ID, SwissProt_acc_Id, SwissProt_entry_Id, or ENSP. The cleaned protein_mutation sheet stored protein change values using fields such as Protein_Change, HGVSp_Short, or HGVSp, but only when both a usable protein identifier and protein change value were available. Many extra mutation annotation columns were removed, including sequencing center fields, dbSNP fields, validation fields, allele depth fields, COSMIC, DrugBank, GO, PolyPhen, SIFT, and ExAC annotations. This information was unnecessary for the project.

The mRNA expression file was cleaned as a wide matrix first and then prepared for long format SQL loading. The original file had genes as rows and TCGA samples as columns, with identifier columns such as Hugo_Symbol and Entrez_Gene_Id. The HUGO symbol, Entrez gene ID, and sample columns were kept because they were needed to connect the data to the gene, sample, and mRNA_expression tables. Gene rows with missing HUGO symbols were removed because they could not connect cleanly to the gene table. Duplicate gene identifiers were checked, and HUGO symbol was not used as the primary key because some symbols appeared more than once or had different Entrez IDs. Expression values were kept as numeric decimal values, and zero values were kept because they can represent meaningful expression measurements rather than missing data. For the final database, the matrix was converted into long format so each row represented one sample gene expression value.

The copy number alteration file was cleaned in a similar way. The original CNA file was also a wide matrix, with genes or features as rows and TCGA samples as columns. The cleaned version kept Hugo_Symbol, Entrez_Gene_Id, Cytoband, and all sample columns. Cytoband was kept because it was included in the final gene table. CNA values such as -2, -1, 0, 1, and 2 were kept as meaningful integer copy number calls. Like the mRNA data, the CNA matrix was converted into long format for SQL loading.

The RPPA protein expression file was cleaned by separating protein or antibody feature information from repeated expression measurements. The original file contained 171 RPPA feature rows and 193 TCGA sample columns. The main identifier, Composite.Element.REF, contained combined values such as YWHAE|14-3-3_epsilon-M-C, so it was split into gene target information and a cleaner protein/RPPA label. The RPPA expression values were converted from wide matrix format into long format, producing 33,003 protein expression records with sample_id, protein_id, and an RPPA value. Negative RPPA values were kept because they are meaningful in normalized protein expression data. For RPPA features that mapped to multiple genes, a separate protein gene relationship sheet was created instead of forcing the feature to link to only one gene.

The gene panel matrix file was cleaned to describe which samples had mutation profiling and GISTIC/CNA profiling available. The original file had 577 sample rows and three columns: SAMPLE_ID, mutations, and gistic. Profile availability was converted into clean 1/0 fields, where samples with a listed platform such as WXS were marked as profiled and samples with NA were marked as not profiled. In the final database, this information was simplified into the sequencing_panel table, with one row per sample showing mutation and GISTIC/CNA profiling availability.

Database Design Decisions

The final database design separates the major clinical and molecular concepts into related tables. The biggest goal was to reduce unnecessary repetition while still keeping the database easy to query. The final schema includes tables for patients, cancer types, samples, genes, mutations, expression values, copy number values, proteins, protein expression values, protein mutation relationships, and sequencing panel information.

The first major design decision was separating patients and samples. A patient and a sample are related, but they are not the same thing. The patient table stores patient level information, while the sample table stores tumor/sample specific information. This makes the database more accurate because most of the molecular measurements are linked to samples rather than directly to patients.

Cancer type was also separated into its own table. Even though this project only uses one cancer type, putting cancer type information in cancer_type prevents repeated text in the sample table and makes the database easier to expand in the future. If another cancer type were added later, the model would already support that structure.

The gene table was created as a central reference table because genes appear in several different datasets. Gene information is needed for mutation data, mRNA expression, copy number alteration data, and protein data. By storing genes once in a central table, the database avoids repeating gene annotations across all the molecular tables. The gene table stores HUGO symbols, Entrez gene IDs, chromosome, gene start and end positions, and cytoband information.

The mutation design changed during the project. In an earlier version, the mutation table included `sample_id`, but this did not fit the data as well. The final design keeps mutation level information in the mutation table and uses `mutation_sample` to connect mutations to samples. This is a better structure because the same mutation can appear in more than one sample. The final design also includes `mutation_gene`, which connects mutations to genes. This keeps gene information separate from mutation information while still keeping the relationship between them.

The copy number table was another important design change. At one point, the copy number table included `mutation_id`, but that did not match the actual structure of the CNA file. The CNA file stores copy number values by sample and gene. It does not store copy number values by mutation. The final `copy_number` table therefore uses `sample_id` and `gene_id`, which better reflects the source data.

The mRNA expression and copy number tables were designed as long format measurement tables. This means each row stores one measurement for one sample feature pair. This design is much better for SQL than the original matrix format because users can filter, join, group, and aggregate values using normal SQL queries. The `mrna_expression` table links to both sample and gene, and the `copy_number` table does the same.

Protein data were separated into multiple tables because protein definitions, protein expression values, and protein level mutation changes are different types of information. The protein table defines each protein, the `protein_quant` table stores RPPA expression values, and the `protein_mutation` table connects proteins to mutations while storing protein change annotations. This structure keeps the database cleaner than putting all protein related fields into one table.

The `sequencing_panel` table was kept separate because it describes profiling availability for each sample. Since each sample has one row of panel information, `sample_id` works as both the primary key and the foreign key.

Normal Form and Practical Deviations

The database was designed to be mostly normalized. Patient information is separated from sample information, cancer type is separated from sample records, gene information is stored in a central reference table, and many to many relationships are done through bridge tables. The mRNA and copy number matrix files were converted into long format so that the database would not have hundreds of sample columns. These decisions help reduce repeated information and make the structure easier to understand.

There were also some practical decisions that are not the only possible design choice. For example, the large molecular tables use surrogate primary keys such as `expression_id`, `cna_id`, and `protein_quant_id`. A composite key such as `sample_id` and `gene_id` or `sample_id` and `protein_id` could also identify rows in some of these tables. However, surrogate keys made the SQL loading process more consistent across tables and easier to manage in phpMyAdmin.

The sample table also contains several sample annotation fields, including expression subtype, methylation status, IDH1 mutation status, tumor mutational burden, and survival fields. Some of these could theoretically be split into additional lookup tables. For this project, I kept them in sample because they directly describe the sample and are easier to query together. This was a practical choice that kept the database understandable without overcomplicating the schema.

Relationships and Constraints

The database uses primary keys and foreign keys to store relationships between tables. The patient table uses `patient_id` as its primary key, and the sample table uses `sample_id` as its primary key. The sample table links to patient through `patient_id` and to cancer_type through `cancer_type_id`. The gene table uses an internal `gene_id`, which is referenced by the mRNA expression, copy number, mutation gene, and protein tables.

The mutation table uses `mutation_id` as its primary key. The `mutation_sample` table connects mutations to samples using `mutation_id` and `sample_id`, while the `mutation_gene` table connects mutations to genes using `mutation_id` and `gene_id`. This structure allows mutations to be connected to samples and genes without forcing all of that information into the mutation table.

The `mrna_expression` table links each expression value to a sample and gene. The `copy_number` table also links each copy number value to a sample and gene. The protein table links proteins to genes, and `protein_quant` links protein expression values to both samples and proteins. The `protein_mutation` table connects protein records to mutation records and stores the protein change annotation. Finally, `sequencing_panel` links back to sample using `sample_id`.

Before writing the final SQL population file, the Python population script checked several relationships to make sure child records had matching parent records. For example, it checked that `protein_quant` protein IDs existed in protein, that mRNA and CNA gene IDs existed in gene, and that sample IDs in molecular tables existed in sample. These checks helped catch relationship problems before importing the SQL into phpMyAdmin.

Script Map

The repository includes the scripts needed to recreate the whole project. The first important script is `02_Converting Matrix to Long Format.py`. This script converts the cleaned mRNA expression and copy number alteration matrix files into long format. The original matrix files had genes as rows and samples as columns. The script reshapes those files so each row

represents one sample gene measurement. The expected outputs are `long_tcga_gbm_mrna_expression.csv` and `long_tcga_gbm_copy_number.csv`.

The second major script is `03_Create SQL Table Population File.py`. This script reads the cleaned datasets and creates the large SQL file used to populate the MySQL database. The output is `04_populate_tcga_gbm_tables.sql`. This file contains insert statements for all final database tables. Because it contains millions of insert rows, it was too large to upload directly to GitHub and is stored externally through the link in `sql/README.md`.

The schema creation script is `01_create_tcga_gbm_tables.sql`. This file creates all tables, primary keys, foreign keys, and relationships in MySQL. It should be run in phpMyAdmin before the population SQL file is imported.

The Neo4j part uses `05_neo4j_diff_exp.sql` and `06_neo4j_cypher_visualization.txt`. These files contain the Cypher code used to load the SQL gotten differential expression results into Neo4j. The Neo4j graph follows the structure Gene to HIGHER_IN relationship to Subtype, where each gene is connected to the GBM subtype where it had higher mean expression.

Reproduction Instructions

To reproduce the database, the first step is to download the raw and cleaned datasets from the external links in `data/raw/README.md` and `data/cleaned/README.md`. These files should be placed into the expected local project folders before running the Python scripts.

Next, open phpMyAdmin, create or select the project database, and run `sql/01_create_tcga_gbm_tables.sql`. This creates the empty MySQL schema with all tables, primary keys, foreign keys, and relationships.

After the schema is created, run `python scripts/02_Converting Matrix to Long Format.py`. This creates the long format mRNA and copy number files needed for loading expression and CNA data into the database. The expected outputs are `long_tcga_gbm_mrna_expression.csv` and `long_tcga_gbm_copy_number.csv`.

Then run `python scripts/03_Create SQL Table Population File.py`. This creates the large SQL population file called `04_populate_tcga_gbm_tables.sql`. This file contains the insert statements needed to populate the database tables. Since the file is too large for GitHub, it is stored externally and linked in `sql/README.md`.

After the SQL population file has been generated or downloaded from the external link, import it into phpMyAdmin. Once the import finishes, the row counts should be checked to confirm that the database loaded correctly. The expected final row counts are 1 row in `cancer_type`, 2,499,863 rows in `copy_number`, 8,237 rows in `gene`, 727,725 rows in `mrna_expression`, 16,090 rows in `mutation`, 16,090 rows in `mutation_gene`, 16,325 rows in `mutation_sample`, 577 rows in `patient`, 7,667 rows in `protein`, 13,771 rows in `protein_mutation`, 33,003 rows in `protein_quant`, 577 rows in `sample`, and 577 rows in `sequencing_panel`. The total expected row count is 3,340,503 rows across 13 tables.

The Neo4j graph can be recreated by opening Neo4j Browser and running the Cypher script in scripts/06_neo4j_cypher_visualization.txt. This loads neo4j/differentially_expressed_genes.csv and creates Gene nodes, Subtype nodes, and HIGHER_IN relationships.

SQL Dump and Finished Database Export

A full SQL dump of the completed database is stored externally because it is too large to upload directly to GitHub. The dump is linked in sql/README.md and is named database_dump.sql. This dump is a finished database export and can be used as a backup or alternative way to recreate the completed database.

Neo4j Component

The Neo4j component was included to show how SQL obtained results from the relational database could be represented in a graph format. I used SQL to compare mean mRNA expression between Classical and Proneural GBM samples. The resulting candidate differentially expressed genes were exported to a CSV file and loaded into Neo4j. In Neo4j, each Gene node represents a candidate differentially expressed gene, and each Subtype node represents either the Classical or Proneural GBM subtype. A HIGHER_IN relationship connects each gene to the subtype where it had higher mean expression. The graph produced two main subtype clusters, one for Classical and one for Proneural.

Limitations and Future Work

One limitation of this project is that differential expression was done through SQL calculated mean expression differences and log2 fold change between Classical and Proneural samples. This was useful for the Neo4j graph, but it is not the same as performing a formal RNA-seq differential expression analysis. A stronger future version could use DESeq2 or a similar statistical method with normalization, statistical testing, and multiple testing correction.

Another limitation is that several files were too large to store directly in GitHub. The raw files, cleaned files, SQL population file, and SQL dump are stored externally. This keeps the repository readable, but it also means that the external links need to stay active for the workflow to remain fully reproducible. Some gene annotation information was also incomplete in the original files. These values were kept NULL rather than guessed or manually filled in. This is good to keep more true to the source data, but it means that some records do not have complete annotation.

The database could also be improved with the performance of it. The main focus of this project database was design and reproducibility. Future improvements could include adding more indexes, creating SQL views for common queries, adding example analysis queries, or building a

smaller test dataset so the whole workflow could be run quickly without downloading the full large files. Also, the Neo4j graph could be improved or expanded as well. This project only loaded differentially expressed genes and subtype relationships. A more advanced graph could include mutations, proteins, copy number alterations, and sample nodes.

Use of AI Assistance

ChatGPT was used as a tool during parts of the data cleaning, database design, and SQL generation. The ChatGPT conversation links used during the project are included below:

<https://chatgpt.com/share/69fe1faf-0854-83ea-aa8f-6a339171c87c>

<https://chatgpt.com/share/69fe1fc9-0ea0-83ea-9569-e39135dd6c64>

<https://chatgpt.com/share/69fd2b06-dbc0-83ea-bb11-f18d7d37f7d8>

Final Output

The final output of this project is a populated MySQL database with 13 tables and 3,340,503 total rows. The database connects TCGA GBM clinical patient and sample information with gene, mutation, mRNA expression, copy number, protein expression, protein mutation, and sequencing panel data. The project also includes a Neo4j graph showing SQL gotten candidate differentially expressed genes connected to the GBM subtype where each gene had higher mean expression. Overall, this project organizes a complex cancer genomics dataset into a structure that is easier to query, reproduce, and explain.